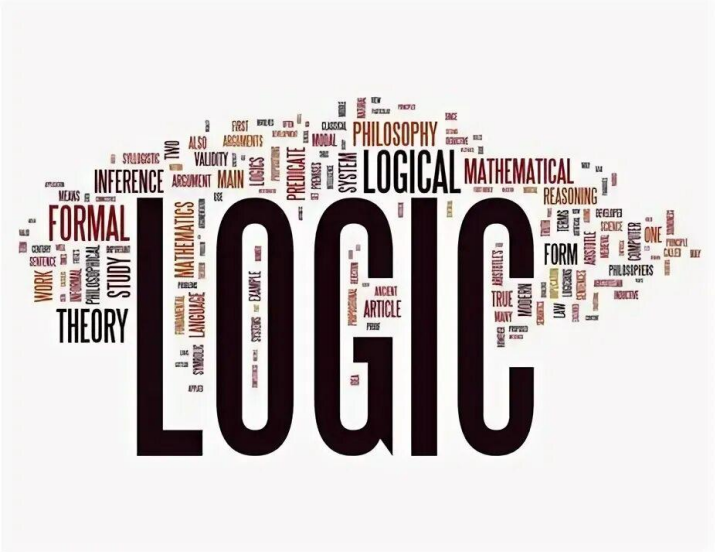


Математическая логика и теория формальных языков

Введение

Математическая логика: ОБЩИЕ ВОПРОСЫ

- *Математическая логика* – наука, изучающая **правильные** методы рассуждений. Что такое “правильность” рассуждения?
- Рассуждение “правильное”, если из истинных знаний, из установленных фактов мы всегда получаем новое истинное знание, которое СООТВЕТСТВУЕТ РЕАЛЬНОСТИ. Если рассуждение “неправильное”, то мы можем из фактов получить ложные, неправильные знания.

- Рассуждения могут быть совершенно о разных вещах, о разных предметах, их свойствах, отношениях между ними.
- **Можно ли в принципе разработать ОБЩУЮ ТЕОРИЮ “правильности” совершенно разных рассуждений?**
- Оказывается, **МОЖНО, НО** только абстрагируясь от смысла, семантики рассуждений, анализируя только их структуры, СХЕМЫ РАССУЖДЕНИЙ.

Курс закладывает базис умения рассуждать и мыслить *согласно правильным схемам*.

Математическая логика: ОБЩИЕ ВОПРОСЫ

Курс представляет собой фундамент **Computer Science (CS)**:

- использование 0 и 1 для кодирования данных, представления дискретной информации и побитовой обработки в компьютерах,
- логика - формальный базис математики, теории и техники CS.

В области ПРОГРАММИРОВАНИЯ **С помощью ЛОГИКИ выражаются**:

- семантика языков программирования,
- спецификация программ – что ДОЛЖНА делать программа,
- верификация программ – выполняет ли программа в точности то, что от нее ожидают.



ЦИТАТЫ

- *“Logic plays a similar role in CS to that played by calculus in the physical science and traditional engineering disciplines” (Moshe Vardi, 2007)*

- *“It is reasonable to hope that the relationship between computation and mathematical logic will be as fruitful in the next centuty as that between analysis and physics in the last” (John MacCarthy, 1961)*



ЦИТАТЫ

- *“Не только вся информатика построена на 0 и 1 и операциях над ними, не только любые конечные структуры данных и аппаратура обработки информации построена на 0 и 1, но 0 и 1 лежат и в основе законов мышления человека, в основе функционирования мозга человека и передачи информации в организме человека”.*

Дональд Кнут, Искусство программирования, т.4 (2008)



Логика - базис компьютерных наук

- Технологии в информатике меняются очень быстро, инженер-программист должен осваивать новые *информационные технологии* каждые 2-5 лет. **Но при этом концептуальные вопросы остаются неизменными**
- Теоретический фундамент, на котором основаны все существующие и будущие информационные технологии, заложен в теоретических курсах, например, в ***дискретной математике*** и ***математической логике***
- Многие понятия базируются знаниях, полученных по дисциплине «Дискретная математика»



Пример «логического умозаключения»

After explaining to a student through various lessons and examples that:

$$\lim_{x \to 8} \frac{1}{x-8} = \infty$$

I tried to check if she really understood that, so I gave her a different example. This was the result:

$$\lim_{x \to 5} \frac{1}{x-5} = \text{ъ}$$



Льюис Кэрролл. «Алиса в стране чудес»

– А что здесь за люди живут? - спросила она.

– Вон там, - сказал Кот и махнул правой лапой, - живет Болванщик. А там, - и он махнул левой, - Мартовский заяц. Все равно, к кому ты пойдешь. Оба не в своем уме.

– На что мне безумцы? - сказала Алиса.

– Ничего не поделаешь, - возразил Кот. - Все мы здесь не в своем уме - и ты, и я.

– Откуда вы знаете, что я не в своем уме? - спросила Алиса.

– Конечно, не в своем, - ответил Кот. - Иначе как бы ты здесь оказалась?

Довод этот, показался Алисе совсем не убедительным, но она не стала спорить, а только спросила:

– А откуда вы знаете, что вы не в своем уме?

– Начнем с того, что пес в своем уме. Согласна?

– Допустим, - согласилась Алиса.

– Дальше, - сказал Кот. - Пес ворчит, когда сердится, а когда доволен, виляет хвостом. Ну, а я ворчу, когда я доволен, и виляю хвостом, когда сержусь. Следовательно, я не в своем уме.

Литература

Chinese proverb:
**“I hear and I forget,
I see and I remember,
I do and I understand”**







ОСНОВНАЯ

- **Ю.Г. Карпов. Теория автоматов // Питер, 2005**
- С. Клини. Математическая логика // М., “Мир”, 1978
- Ю.В.Ивлев. Логика // Москва, 1998
- С.Д. Шапорев. Математическая логика // БХВ, 2005
- Слайды лекций

ДОПОЛНИТЕЛЬНАЯ

- **Ю.Г.Карпов. Model checking. Верификация параллельных и распределенных программных систем // СПб, БХВ, 2010**
- Р. Смаллиан. Принцесса или тигр // М., “Мир”, 1985
- Ч.Чень, Р.Ли. Математическая логика и автоматическое доказательство теорем // М., “Мир”, 1983
- С. Рассел, П. Норвиг. Искусственный интеллект. Современный подход // М., “Вильямс”, 2006
- Р.Пенроуз. Новый ум короля // Oxford Univ Press, 1989
- Ф.А. Новиков. Дискретная математика для программистов

Структура курса

- Двоичные функции
- Математическая логика
- Темпоральные логики
- Model checking (верификация)

- ТЕОРИЯ АЛГОРИТМОВ



Структура курса

ДВОИЧНЫЕ ФУНКЦИИ

- Двоичные функции - функции алгебры логики
- Двоичные функции – формы представления, минимизация
- Двоичные функции – базисы
- Применения двоичных функций
- Двоичные функции – BDD
- Конечные автоматы



Двоичные функции ≡ булевы (логические) функции

- Двоичная система счисления была придумана в XVII веке Готфридом Лейбницем
- Двоичные функции называют булевыми функциями (БФ): впервые такие функции были использованы Джорджем Булем при формализации законов логики



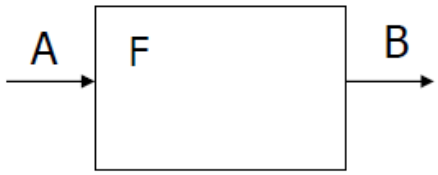
- Джордж Буль (1815-1864) - английский математик и логик, один из основоположников математической логики.
- Разработал алгебру логики (теорию двоичных функций) в трудах:
  - "Математический анализ логики" (1847)
  - "Исследование законов мышления" (1854)

Теория двоичных функций составила основу формальной логики

- Но двоичные функции имеют отношение НЕ ТОЛЬКО К ЛОГИКЕ, они имеют и самостоятельное значение (и множество применений), как частный случай отображений конечных множеств вида
- $F : \{0,1\}^n \rightarrow \{0,1\}$ (т.е. отображений с n аргументами, имеющими значения 0 или 1, и результатами, имеющими значения из той же области)

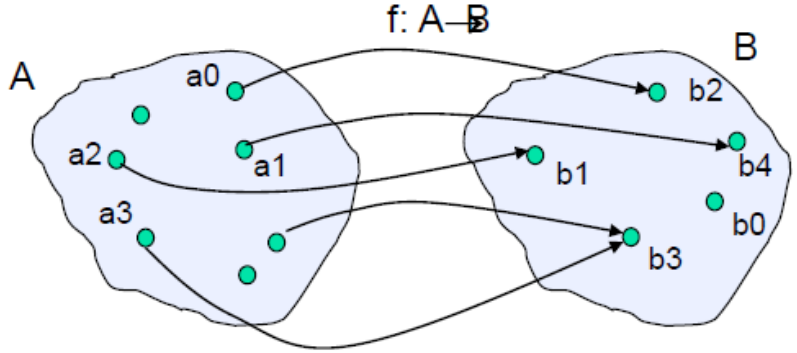
В этой части курса мы их исследуем двоичные функции просто как конечные отображения

Конечные функциональные преобразователи



A, B – конечные множества

- Конечные функциональные преобразователи реализуют функциональное отображение конечных множеств
- Функция – отношение двух множеств A и B (т.е. множество пар, первый элемент – из первого множества, второй элемент – из второго множества)
  - Обозначается f: A→B
- Функциональное отношение - элементу первого множества ставится в соответствие ровно один элемент второго



x	f(x)
a0	b2
a1	b4
a2	b1
a3	b3
a4	b3

- Частичные функции
- Функции ‘на’ и ‘в’
- Изоморфизм
- Конечные функции (задаются таблицей)

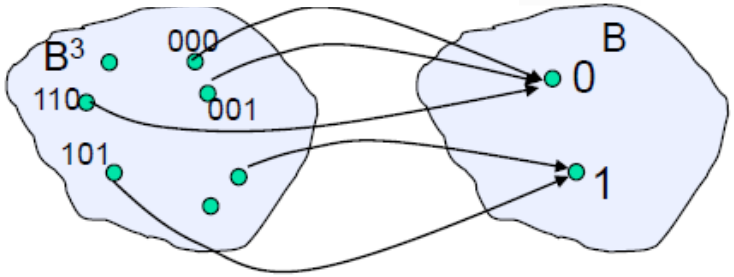
- Иногда функции “частичные” – для некоторых аргументов значения функции не определены

Двоичные (булевы) функции - частный случай конечного отображения

Пусть n=3

B={0, 1}

f(0,0,0)=0;
f(1,1,0)=0;
f(1,0,1)=1;
…

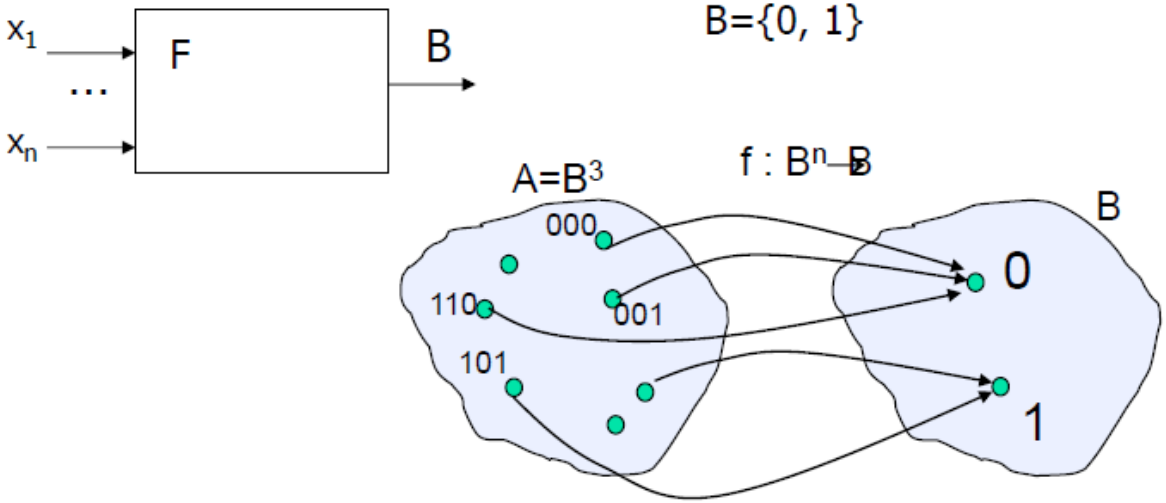


- Двоичные функции были исследованы Джорджем Булем для формализации логических рассуждений. Это определило их специфическое название:
  "логические функции" или "булевы функции"
- С точки зрения математики, двоичные функции – это просто функции от одного или нескольких аргументов, принимающие только два значения, 0 и 1, и сами функции тоже принимают значения 0 или 1.
  В этом смысле двоичные функции мало отличаются от троичных и вообще от любых других конечных функциональных отображений

Специфика булевых функций - та, что они оперируют ТОЛЬКО с 0 и 1 - определяет огромное число возможных приложений

Двоичные функции – отображения вида $\{0,1\}^N \rightarrow \{0, 1\}$

- двоичные функции от n двоичных переменных являются частным случаем конечных функциональных отображений



ТИ

$x_1x_2x_3$	f
0 0 0	0
0 0 1	0
0 1 0	0
0 1 1	1
1 0 0	1
1 0 1	1
1 1 0	0
1 1 1	0

Таблица двоичной функции называется таблицей истинности (ТИ)

Поскольку результат двоичной функции – 0 или 1, то любая двоичная функция может быть аргументом другой двоичной функции (суперпозиция функций: подстановка функции в качестве аргумента)

Любые конечные функциональные преобразователи можно строить с использованием только БФ

x	F(x)
a0	b2
a1	b0
a2	b1
a3	b3
a4	b3
a5	b1
a6	b2



x	$x_1x_2x_3$	y_1y_2	F(x)
a0	0 0 0	1 0	b2
a1	0 0 1	0 0	b0
a2	0 1 0	0 1	b1
a3	0 1 1	1 1	b3
a4	1 0 0	1 1	b3
a5	1 0 1	0 1	b1
a6	1 1 0	1 0	b2
	1 1 1	- -	



F – вектор <f_1, f_2>

$x_1x_2x_3$	$f_1=y_1$	$f_2=y_2$
0 0 0	1	0
0 0 1	0	0
0 1 0	0	1
0 1 1	1	1
1 0 0	1	1
1 0 1	0	1
1 1 0	1	0
1 1 1	--	--

Кодируем взаимно однозначно двоичными наборами

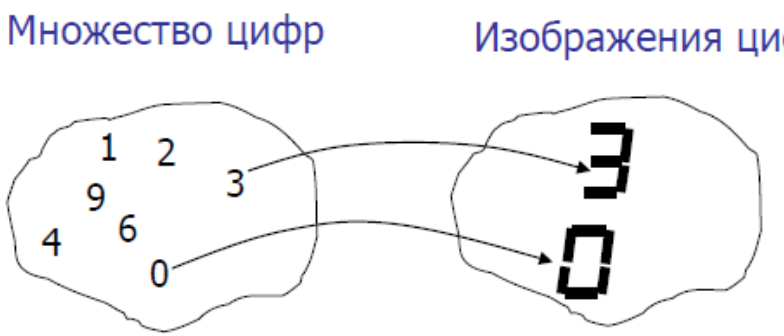
a0 ⇔000 b0 ⇔00
a1 ⇔001 b1 ⇔01
a2 ⇔010 b2 ⇔10
a3 ⇔011 b3 ⇔11
a4 ⇔100
a5 ⇔101
a6 ⇔110

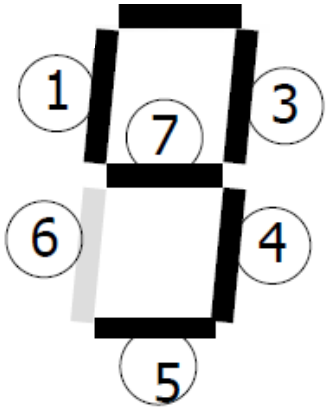
Любое конечное отображение может быть сведено к набору двоичных отображений

Каждая двоичная функция f_1, f_2, … либо может быть реализована отдельно, либо все они могут быть реализованы совместно

Пример: построение схемы отображения электронных часов и калькуляторов







Кодируем

0 ⇔0000
1 ⇔0001
2 ⇔0010
3 ⇔0011
4 ⇔0100
5 ⇔0101
6 ⇔0110
7 ⇔0111
8 ⇔1000
9 ⇔1001

изображение 0 ⇔1111110
изображение 1 ⇔0011000
изображение 2 ⇔0110111
изображение 3 ⇔0111101
изображение 4 ⇔1011001
изображение 5 ⇔1101101
изображение 6 ⇔1101111
изображение 7 ⇔0111000
изображение 8 ⇔1111111
изображение 9 ⇔1111101

	Код цифры	Код изображения цифры			
	$x_1x_2x_3x_4$	f_1	f_2		f_7
0	0 0 0 0	1	1		0
1	0 0 0 1	0	0		0
2	0 0 1 0	0	1		1
3	0 0 1 1	0	1		1
4	0 1 0 0	1	0		1
…	…	…	…		…

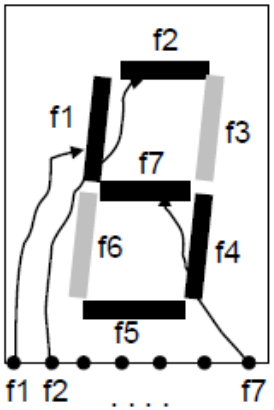
Система управления электронных часов



Как построить схему отображения электронных часов?

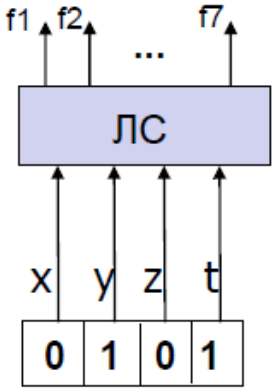
Двоичные функции схемы отображения – частично определенные





	x y z t	f1	f2	f3	f4	f5	f6	f7
0	0 0 0 0	1	1	1	1	1	1	0
1	0 0 0 1	0	0	1	1	0	0	0
2	0 0 1 0	0	1	1	0	1	1	0
3	0 0 1 1	0	1	1	1	0	1	1
4	0 1 0 0	1	0	1	1	0	0	1
5	0 1 0 1	1	1	0	1	1	0	1
6	0 1 1 0	1	1	0	1	1	1	1
7	0 1 1 1	0	1	1	1	0	0	0
8	1 0 0 0	1	1	1	1	1	1	1
9	1 0 0 1	1	1	1	1	1	0	1
	1 0 1 0	-	-	-	-	-	-	-
	1 0 1 1	-	-	-	-	-	-	-
	1 1 0 0	-	-	-	-	-	-	-
	1 1 0 1	-	-	-	-	-	-	-
	1 1 1 0	-	-	-	-	-	-	-
	11 1 1	-	-	-	-	-	-	-

ЛС:
4 входа,
7 выходов



Дополнительные возможности схемы отображения – отображение символов

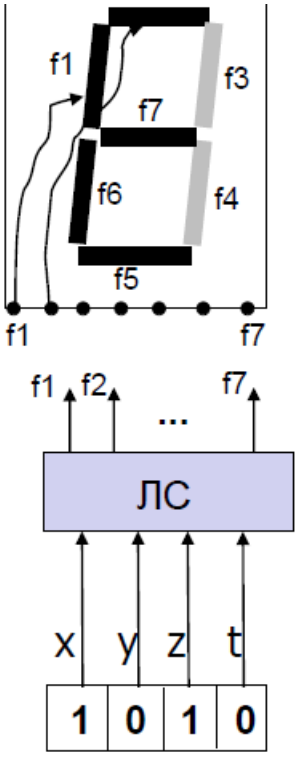
Схема отображения электронного сейфа



4 двоичных разряда – кроме цифр до 6 доп символов

CLOSEd для букв O и S можно использовать цифры 0 и 5

	x y z t	f1	f2	f3	f4	f5	f6	f7
0	0 0 0 0	1	1	1	1	1	1	0
1	0 0 0 1	0	0	1	1	0	0	0
2	0 0 1 0	0	1	1	0	1	1	0
3	0 0 1 1	0	1	1	1	0	1	1
4	0 1 0 0	1	0	1	1	0	0	1
5	0 1 0 1	1	1	0	1	1	0	1
6	0 1 1 0	1	1	0	1	1	1	1
7	0 1 1 1	0	1	1	1	0	0	0
8	1 0 0 0	1	1	1	1	1	1	1
9	1 0 0 1	1	1	1	1	1	0	1
E	1 0 1 0	1	1	0	0	1	1	1
C	1 0 1 1	1	1	0	0	1	1	0
L	1 1 0 0	1	0	0	0	1	1	1
d	1 1 0 1	0	0	1	1	1	1	1
	1 1 1 0	-	-	-	-	-	-	-
	11 1 1	-	-	-	-	-	-	-



ЛС: 4 входа, 7 выходов

Двоичные функции: как их реализовывать, как с ними работать?

$F \equiv G$ – функции F и G равносильны

$x_1\ x_2\ x_3$	F	G
0 0 0	0	0
0 0 1	1	1
0 1 0	1	1
0 1 1	0	0
1 0 0	1	1
1 0 1	0	0
1 1 0	0	0
1 1 1	1	1

Любая двоичная функция определяется таблицей истинности (ТИ), т.е. отношением
<набор значений аргументов> →< значение функции>

Две функции равны (равносильны), если их ТИ совпадают

Мы хотим уметь манипулировать любыми двоичными функциями от разного (большого) числа аргументов, и строить схемы и алгоритмы, которые реализуют такие функции и манипуляции (работу) с ними

Количество двоичных функций

- Сколько всего двоичных функций от n двоичных аргументов?



Сколько двоичных векторов длины **n**?

$$2^n$$

Сколько двоичных векторов длины 2^n?

$$2^{2^n}$$

От 0 переменных – 2

От 1 переменной – 4

От 2 переменных – 16

От 3 переменных – 256

От 4 переменных - 65 536

От 5 переменных – 4 294 967 296

На практике используются функции от десятков и сотен двоичных аргументов

Как с ними работать??

Все двоичные функции можно перенумеровать

$x_1x_2x_3$	F
0 0 0	0
0 0 1	0
0 1 0	0
0 1 1	1
1 0 0	0
1 0 1	0
1 1 0	1
1 1 1	1

Двоичный номер функции

Выписываем значения функции F на наборах

111, 110, 101, 100, 011, 010, 001, 000

Номер функции F = 1 1 0 0 1 0 0 0_2=

$= 3\ 1\ 0_8 = 200_{10}$

Обратно, пусть задан номер функции. Какая это функция?

Пример: функция F, ее № $200_{10} = 3*64 + 1*8 + 0 = 310_8 = 11\ 001\ 000_2$

200_{10}

На функцию можно ссылаться по ее номеру

Двоичные функции. Как подойти к исследованию?

Таблица истинности функции от 100 двоичных переменных содержит $2^{100} \sim 10^{30}$ строк

(миллиард триллионов триллионов строк ТИ)

С такими функциями нельзя работать, используя таблицу истинности

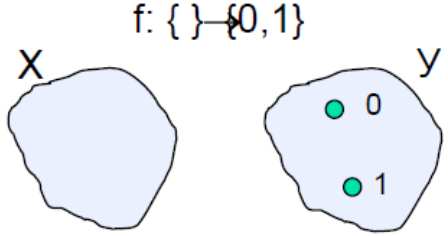
Нужно искать другие формы представления двоичных функций

Будем БФ изучать, начиная с самых простых – от нуля, одной, двух, трех и т.д. переменных. Дойдем и до 100 □переменных

Двоичные функции f: $\{0,1\}^N \to \{0,1\}$ от 0 и 1 аргументов

n=0

- Нульарные функции (от 0 аргументов) – константы, элементы множества {0,1}. Их всего 2 – по числу элементов множества B.



f( ) = 0

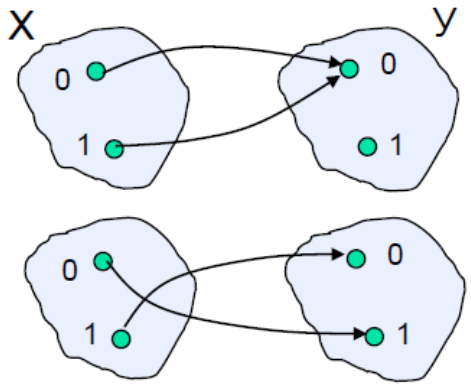
f( ) = 1

Константы 0 и 1

Можно считать, что у этих функций произвольное число аргументов, а значение не зависит от аргументов

n=1

- Унарные функции (от 1 аргумента). Их всего 4



x	$f_1(x)$	$f_2(x)$	$f_3(x)$	$f_4(x)$
0	0	0	1	1
1	0	1	0	1

$f_1(x) = 0$;

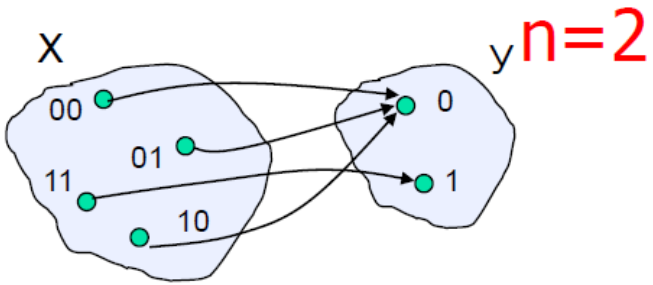
$f_2(x) = x$; Тожд

$f_3(x) = \neg x$; НЕ

$f_4(x) = 1$;

f_3 - отрицание, НЕ

Двоичные функции от 2-х аргументов



"Интересные" функции

x_1x_2	f_0	f_1	f_2	f_3		f_{14}	f_{15}
0 0	0	1	0	1		1	1
0 1	0	0	1	1		1	1
1 0	0	0	0	0		1	1
1 1	0	0	0	0		0	1

x y	x∧y	x∨y	x⇒y	x⊕y	x⇔y	x|y	x↓y
0 0	0	0	1	0	1	1	1
0 1	0	1	1	1	0	1	0
1 0	0	1	0	1	0	1	0
1 1	1	1	1	0	1	0	0

Их все нужно помнить

x∧y – конъюнкция И (x&y)

x∨y – дизъюнкция ИЛИ

x ⇒y - импликация

x⊕y - сложение по модулю 2

x ⇔y – эквивалентность

x|y - штрих Шеффера

x↓y - стрелка Пирса

Знак '≡' будем использовать как отношение равенства (равносильности) функций (часто путают с '⇔– логической операцией)

Обозначения двоичных функций от 0, 1 и 2-х аргументов

0 и 1 - ⊥ и T, *False* и *True*, F и T, *Истина* и *Ложь*

¬x - отрицание, НЕ $f_{\neg}(x)$, Not(x), !x, ~x

x∧y - конъюнкция, И (логическое умножение)
$f_{\&}(x,y)$, And(x,y), xy, x&y, min(x,y), x&&y – побитовая операция

x∨y - дизъюнкция ИЛИ (логическое сложение) Or(x,y), x||y, x+y, max (x,y)

x⇒y - импликация, IMP, если x то y, Imp(x,y), x→y

x ⊕y - сложение по модулю 2, неравнозначность, искл ИЛИ, XOR, x <> y

x ⇔y - эквивалентность, Equ, равнозначность, iff Знак '≡' - равнозначность

x|y - штрих Шеффера NAND (В языке Си '|' означает побитовое ИЛИ !!!)

x↓y - стрелка Пирса NOR

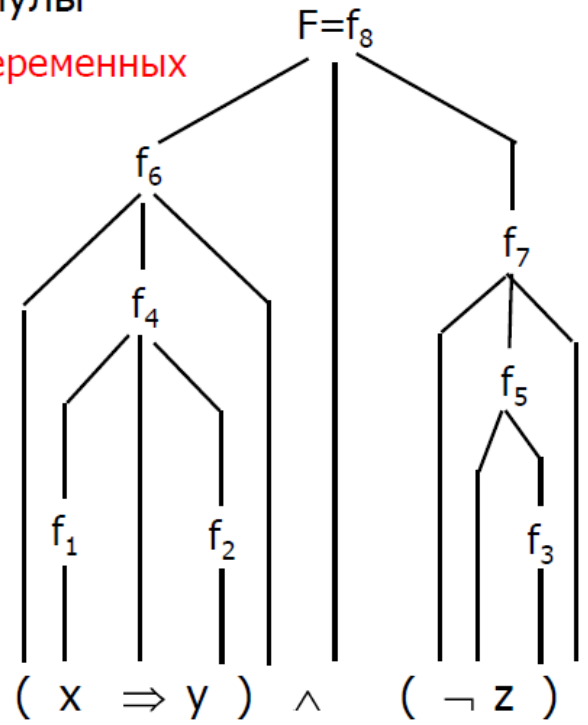
В программировании: Basic: Not, And, Or, Xor, Imp
C: ! (для НЕ), &&, ||, | (побитовое 'или'), &(побитовое И)
Mathcad: ¬, ∧, ∨, ⊕

Символ & (амперсанд) введён Джорджем Булем.
Символ & стал логотипом математической логики

Формальное определение двоичной формулы

Является ли $F=(x \vee y) \Rightarrow (\neg z)$ двоичной формулой?
Это можно проверить, строя синтаксическое дерево на основе формального определения формулы

- Задано (бесконечное) множество двоичных переменных
- Двоичная формула определяется рекурсивно:
  - любая двоичная переменная является двоичной формулой;
  - если A – двоичная формула, то (A) – тоже двоичная формула;
  - если A – двоичная формула, то ¬A – тоже двоичная формула;
  - если A и B – двоичные формулы, то:
    - $A \wedge B$ – тоже двоичная формула;
    - $A \vee B$ – тоже двоичная формула;
    - $A \Rightarrow B$ – тоже двоичная формула;
    - $A \oplus B$ – тоже двоичная формула;
    - …



синтаксическое дерево формулы F

Значение формулы: структурная индукция

- Интерпретаця двоичной формулы – это отображение
  σ: $\{x_1, x_2, \dots x_n\} \to \{0,1\}$

  σ - каждой переменной присваивается конкретное значение, либо 0, либо 1
- Значение (оценка, оценивание) формулы f при интерпретации σ (обозначается $I_\sigma(f)$) определяется по структуре формулы f:

Оценивание строится по таблицам истинности

$I_\sigma(x_i)$	=	$\sigma(x_i)$;	оценивание переменной – это ее значение
$I_\sigma(\ (A)\ )$	=	$I_\sigma(A)$;	взятие формулы в скобки не меняет ее значения
$I_\sigma(\neg A)$	=	$\neg\, I_\sigma(A)$;	знак отрицания перед формулой меняет ее значение: оно равно отрицанию значения формулы
$I_\sigma(A \wedge B)$	=	$I_\sigma(A) \wedge I_\sigma(B)$;	...
$I_\sigma(A \vee B)$	=	$I_\sigma(A) \vee I_\sigma(B)$;	...
$I_\sigma(\ A \oplus B)$	=	$I_\sigma(A) \oplus I_\sigma(B)$;	...
$I_\sigma(\ A \Leftrightarrow B)$	=	$I_\sigma(A) \Leftrightarrow I_\sigma(B)$;	...
$I_\sigma(\ A \Rightarrow B)$	=	$I_\sigma(A) \Rightarrow I_\sigma(B)$;	...

Пример вычисления значения формулы на основе структурной индукции (по синтаксическому дереву)

Пример. При интерпретации $\sigma=(x\leftarrow 1, y\leftarrow 0, z\leftarrow 0)$ найти значение формулы

$(x \Rightarrow y) \wedge (\neg z)$

$I_\sigma(x) = \sigma(x)$; оценивание формулы

$\sigma(x)=x$ - это интерпретация (значение) x

$I_\sigma((A)) = I_\sigma(A)$;

$I_\sigma(\neg A) = \neg I_\sigma(A)$;

$I_\sigma(A \wedge B) = I_\sigma(A) \wedge I_\sigma(B)$;

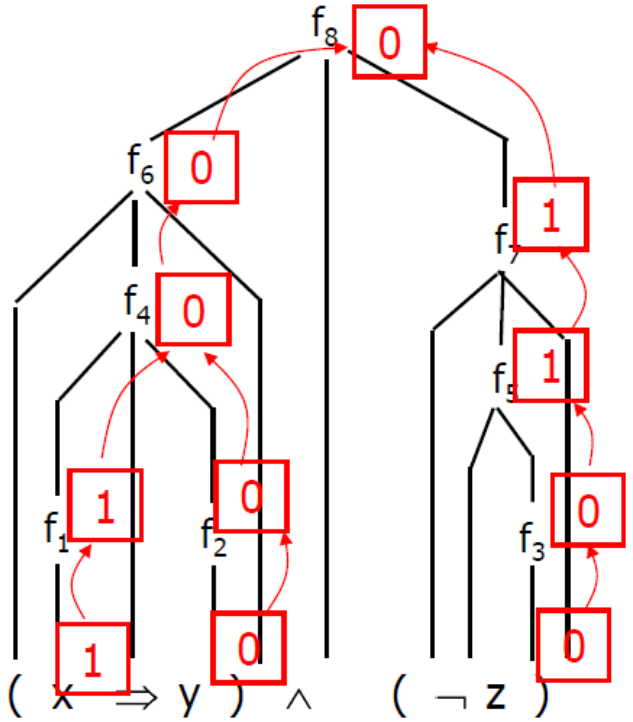
$I_\sigma(A \vee B) = I_\sigma(A) \vee I_\sigma(B)$;

$I_\sigma(A \Rightarrow B) = I_\sigma(A) \Rightarrow I_\sigma(B)$;

...

$f_1 = x$;	$I_\sigma(f_1) = 1$
$f_2 = y$;	$I_\sigma(f_2) = 0$
$f_3 = z$;	$I_\sigma(f_3) = 0$
$f_4 = f_1 \Rightarrow f_2$;	$I_\sigma(f_4) = 0$
$f_5 = \neg f_3$;	$I_\sigma(f_5) = 1$
$f_6 = (f_4)$;	$I_\sigma(f_6) = 0$
$f_7 = (f_5)$;	$I_\sigma(f_7) = 1$
$f_8 = f_6 \wedge f_7$;	$I_\sigma(f_8) = 0$

Структурная индукция – снизу вверх по синтаксическому дереву формулы



Формулы без скобок: правило приоритетов

$\neg x \wedge y \oplus z \vee x \Leftrightarrow y \vee t$ - Как понимать?

x y	xy	x∨y	x⇒y	x⊕y	x⇔y	x|y	x↓y
0 0	0	0	1	0	1	1	1
0 1	0	1	1	1	0	1	0
1 0	0	1	0	1	0	1	0
1 1	1	1	1	0	1	0	0

$\neg x \quad \wedge \quad y \quad \oplus \quad z \vee x \quad \Leftrightarrow y \vee t$

$(((\neg x) \quad \wedge \quad y) \quad \oplus \quad (\ z \vee x \ )) \Leftrightarrow (y \vee t)$

В сомнительных случаях используйте скобки

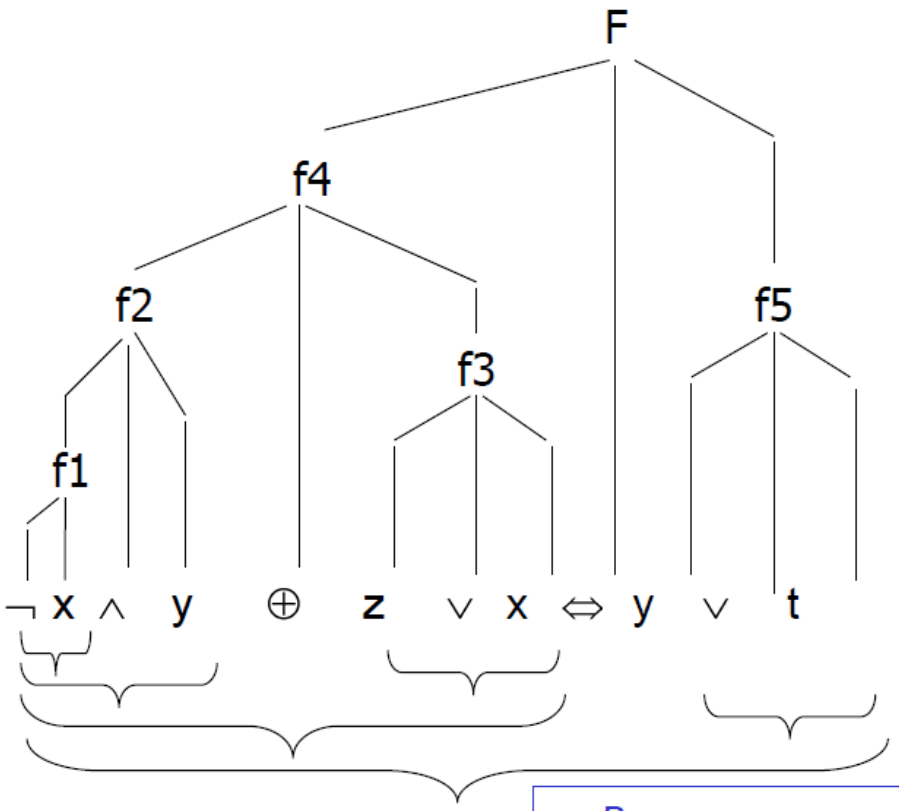
Приоритеты:

- $\neg$ **НЕ**
- $\wedge$ **И**
- $\vee$ **ИЛИ**
- $\Rightarrow \oplus \Leftrightarrow$ **... Все остальные приоритеты одинаковы**

Существуют и другие правила приоритетов

Порядок вычисления значения формулы определяется синтаксическим деревом формулы

$\neg x \wedge y \oplus z \vee x \equiv y \vee t$ - Как понимать?



f1 = ¬x
f2 = f1 ∧ y
f3 = z ∨ x
f4 = f2 ⊕f3
f5 = y ∨ t
F = f4 ⇔f5

Приоритеты:

¬ **НЕ**

∧ **И**

∨ **ИЛИ**

⇒⊕ ⇔... **Все остальные приоритеты одинаковы**

Существуют и другие правила приоритетов

Всегда вычисляем последовательно по дереву

Как построить ТИ булевой функции по дереву

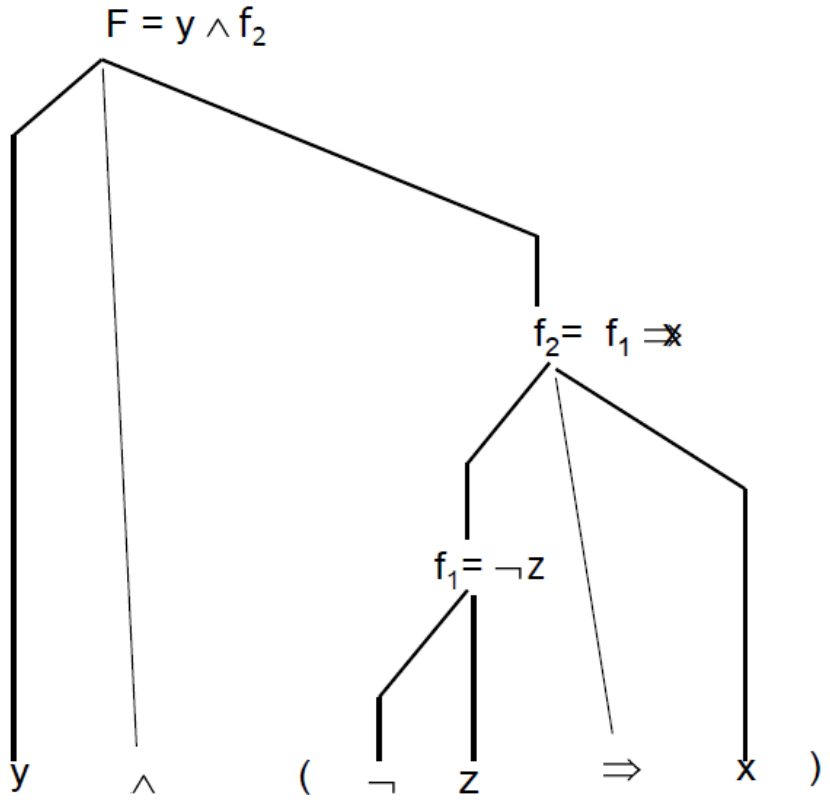


Таблица истинности БФ

$F = y \wedge (\neg z \Rightarrow x)$

x y z	$f_1 = \neg z$	$f_2 = f_1 \Rightarrow x$	$F = y \wedge f_2$
0 0 0	1	0	0
0 0 1	0	1	0
0 1 0	1	0	0
0 1 1	0	1	1
1 0 0	1	1	0
1 0 1	0	1	0
1 1 0	1	1	1
1 1 1	0	1	1

Типы булевых функций

- Типы БФ: невыполнимые, выполнимые, тавтологии (общезначимые)

$x_1x_2x_3$	f1	f2	f3
0 0 0	0	0	1
0 0 1	0	0	1
0 1 0	0	1	1
0 1 1	0	0	1
1 0 0	0	0	1
1 0 1	0	0	1
1 1 0	0	1	1
1 1 1	0	1	1

НЕВЫПОЛНИМАЯ (противоречивая)–
на всех значениях аргументов f равна 0

ВЫПОЛНИМАЯ –
есть значения аргументов, на которых f = 1

ТАВТОЛОГИЯ (общезначимая) –
на всех значениях аргументов f =1

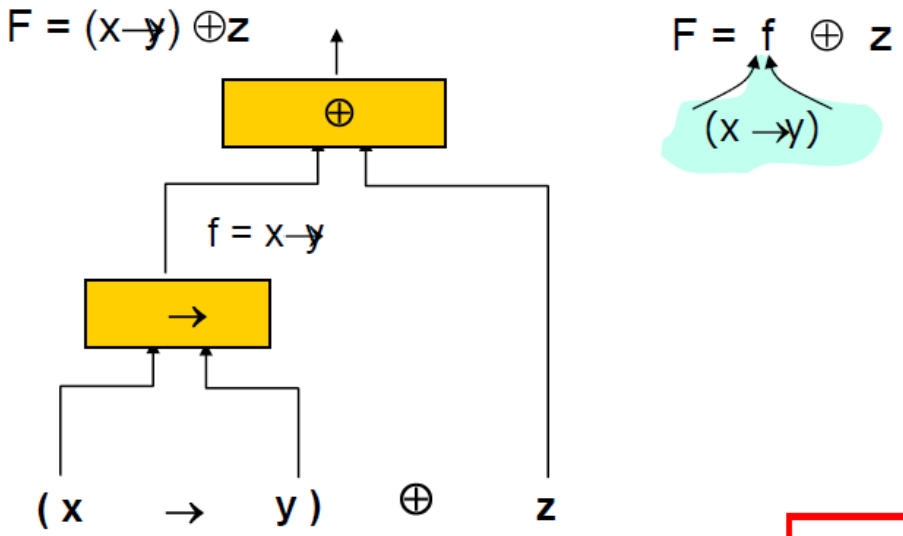
НЕОБЩЕЗНАЧИМАЯ –
есть значения аргументов, на которых f = 0

Проверка того, является ли функция выполнимой, является важнейшей проблемой. Она называется проблемой SAT (Satisfiability)

ВОПРОС. Как, используя алгоритм SAT, проверить не только выполнимость, но и невыполнимость, общезначимость и НЕобщезначимость функции?

Суперпозиция двоичных функций

- Каждый аргумент двоичной функции - двоичный, поэтому вместо любого аргумента любой двоичной функции можем подставить другую функцию, дающую как результат 0 или 1



x y z	f = x→y	F=f⊕z
0 0 0	1	1
0 0 1	1	0
0 1 0	1	1
0 1 1	1	0
1 0 0	0	0
1 0 1	0	1
1 1 0	1	1
1 1 1	1	0

Функции от N переменных можем представить, как суперпозицию более простых функций
F(x, y, z) = (x →y) ⊕z

Суперпозиция каких простых функций может дать любую двоичную функцию?
Как строить разложение двоичных функций на "простые"?

Двоичные функции могут представляться РАЗНЫМИ формулами

- Определение. Две функции называются равносильными, если их таблицы истинности совпадают

x y z	$f_1 = x \to y$	$F = f_1 \oplus z$
0 0 0	1	1
0 0 1	1	0
0 1 0	1	1
0 1 1	1	0
1 0 0	0	0
1 0 1	0	1
1 1 0	1	1
1 1 1	1	0

$F(x, y, z) = (x \to y) \oplus z$

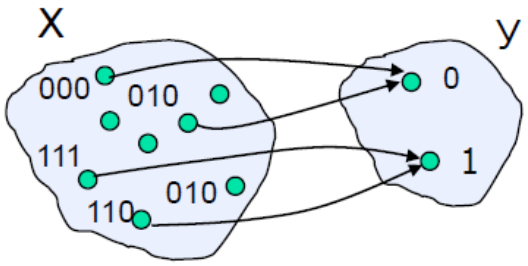
x y z	$\phi_1 = \neg x \neg z$	$\phi_2 = x(y \oplus z)$	$\Phi = \phi_1 \vee \phi_2$
0 0 0	1	0	1
0 0 1	0	0	0
0 1 0	1	0	1
0 1 1	0	0	0
1 0 0	0	0	0
1 0 1	0	1	1
1 1 0	0	1	1
1 1 1	0	0	0

$\Phi(x, y, z) = \neg x \neg z \vee x(y \oplus z)$

$\Phi \equiv F$: Равносильные функции, хотя они представлены совершенно различными по виду формулами

Двоичные функции от 3-х аргументов

n=3



$x_1x_2x_3$	0	f_1	ite	$\wedge$	$\vee$	maj	⋯
0 0 0	0	0	0	0	0	0	
0 0 1	0	0	1	0	1	0	
0 1 0	0	1	0	0	1	0	
0 1 1	0	0	1	0	1	1	
1 0 0	0	0	0	0	1	0	
1 0 1	0	0	0	0	1	1	
1 1 0	0	1	1	0	1	1	
1 1 1	0	1	1	1	1	1	

Всего функций $2^8 = 256$

“Интересные” функции

$ite(x_1, x_2, x_3)$ = ***if*** x_1 ***then*** x_2 ***else*** $x_3 \equiv x_1x_2 \vee \neg x_1x_3$

$maj(x_1, x_2, x_3)$ = majority $(x, y, z) \equiv xy \vee yz \vee xz \equiv (x \vee y)(y \vee z)(z \vee x)$ (медиана)

…

Нужно ли изучать функции от 3-х, 4-х и более переменных?

Пример: задача проверки равносильности формул

- Как проверить, что две булевы функции эквивалентны (равносильны?

Например, $f_1(P, Q, R) = P \vee Q \vee R$

И

$f_2(P, Q, R) = (P \wedge \neg Q) \vee (Q \wedge \neg R) \vee (R \wedge \neg P) \vee (P \wedge Q \wedge R)$

эквивалентны?

Можно построить таблицы истинности этих функций и сравнить их: для каждого набора значений переменных P, Q, R проверить, совпадут ли значения $f_1(P, Q, R)$ и $f_2(P, Q, R)$. Всего таких строк - 8

Вопрос: как проверить равносильность булевых функций от 10 аргументов? От 30 аргументов? От 50 аргументов?

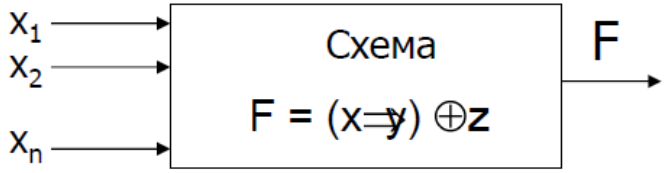
Число строк ТИ функции от 50 аргументов равно $2^{50} \approx 10^{15}$ - миллион миллиардов строк

Можно ли проверить равносильность функций по их формулам?

НЕТ!

Формулы не являются КАНОНИЧЕСКИМ представлением функции

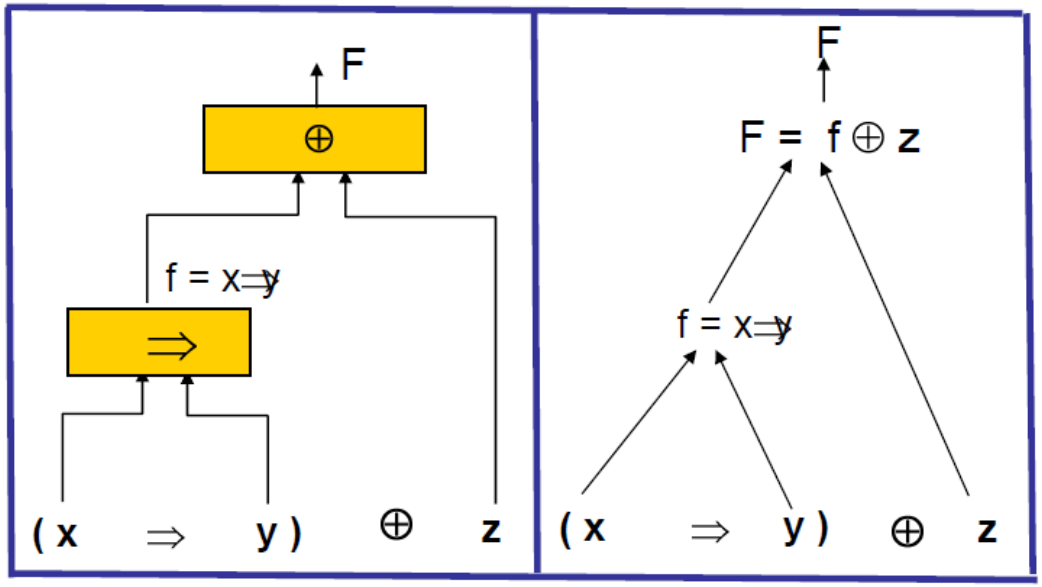
Аппаратная реализация двоичных функций



$F = f \oplus z$

$(x \Rightarrow y)$

Что внутри схемы?



x y z	f = x⇒y	F=f⊕z
0 0 0	1	1
0 0 1	1	0
0 1 0	1	1
0 1 1	1	0
1 0 0	0	0
1 0 1	0	1
1 1 0	1	1
1 1 1	1	0

Реализация повторяет синтаксическое дерево (структуру функции)

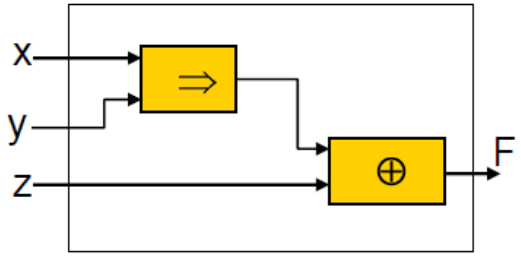
F- функция от 3х переменных, но она представлена суперпозицией двух функций от 2х переменных

Суперпозиция двоичных функций

- Если любую БФ можно представить суперпозицией функций множества M, то M называется функционально полным множеством.
- Минимальное функционально полное множество называется базисом

$F = (x \Rightarrow y) \oplus z$

$= f \oplus z$, где $f = x \Rightarrow y$



Для реализации F можно иметь два блока,
блок, реализующий ⇒
блок, реализующий ⊕

x y z	f = x⇒y	F = f ⊕ z
0 0 0	1	1
0 0 1	1	0
0 1 0	1	1
0 1 1	1	0
1 0 0	0	0
1 0 1	0	1
1 1 0	1	1
1 1 1	1	0

Для реализации любой БФ достаточно иметь только блоки, реализующие функции базиса

Можно ли любую двоичную функцтю построить суперпозицией функций { ⇒⊕}?

Существует ли хотя бы один базис?

Основная теорема теории двоичных функций

- Теорема. Любая булева функция может быть представлена как суперпозиция только трех булевых функций: И, ИЛИ, НЕ

Доказательство 1 (неформальное)

x y z	ite	f_1	f_2	f_3	f_4
0 0 0	0	0	0	0	0
0 0 1	1	1	0	0	0
0 1 0	0	0	0	0	0
0 1 1	1	0	1	0	0
1 0 0	0	0	0	0	0
1 0 1	0	0	0	0	0
1 1 0	1	0	0	1	0
1 1 1	1	0	0	0	1

Построим двоичные функции, принимающие значение 1 только на одном наборе аргументов

$f_1 = \neg x \neg y z;$

$f_2 = \neg x y z;$

$f_3 = x y \neg z;$

$f_4 = x y z$

$ite(x, y, z) = f_1 \vee f_2 \vee f_3 \vee f_4$

$ite(x, y, z) = \neg x \neg y z \vee \neg x y z \vee x y \neg z \vee x y z$

Свойства булевых функций

- Свойства бинарных операций:
  - Ассоциативность операции ∇: $(x \nabla y) \nabla z \equiv x \nabla (y \nabla z)$ – пишем $x \nabla y \nabla z$
  - Коммутативность операции ∇: $x \nabla y \equiv y \nabla x$
  - Дистрибутивность двух бинарных операций

  Дистрибутивность бинарной операции ∇ относительно бинарной операции $\otimes$

  слева: $x \nabla (y \otimes z) \equiv (x \nabla y) \otimes (x \nabla z)$

  справа: $(y \otimes z) \nabla x \equiv (y \nabla x) \otimes (z \nabla x)$
  - Очевидно, что в обычной алгебре умножение дистрибутивно относительно сложения как слева, так и справа

Например, свойство дистрибутивности импликации относительно дизъюнкции справа выполняется, если равносильны две функции:

$f1 = (a \vee b) \Rightarrow c$ и $f2 = (a \Rightarrow c) \vee (b \Rightarrow c)$ – Как проверить? По ТИ

А какие свойства у других булевых функций?
Например, ассоциативна ли импликация? Коммутативна ли дизъюнкция?
Все это можно проверить с помощью построения таблиц истинности

Свойства булевых функций

Для выполнения эквивалентных преобразований с булевыми функциями нужно знать их свойства (эквивалентные соотношения)

x y	$x \wedge y$	$x \vee y$	$x \oplus y$	$x \Leftrightarrow y$
0 0	0	0	0	1
0 1	0	1	1	0
1 0	0	1	1	0
1 1	1	1	0	1

Существует множество красивых соотношений

Свойства $\oplus$

$1 \oplus x \equiv \neg x$ $\quad\quad$ $x \oplus y \equiv y \oplus x$

$x \oplus x \equiv 0$ $\quad\quad$ $\neg(x \oplus y) \equiv x \Leftrightarrow y$

$(x \oplus y) \oplus x \equiv y$ $\quad\quad$ $(x \oplus y) \oplus y \equiv x$

Соотношения конъюнкции, дизъюнкции и сложения по модулю 2

$x \wedge y \equiv x \oplus y \oplus (x \vee y)$

$x \vee y \equiv x \oplus y \oplus (x \wedge y)$

$x \oplus y \equiv (x \vee y) \wedge \neg(x \wedge y)$

…

…

…

Свойства булевых функций И, ИЛИ, НЕ

- Наиболее часто используются свойства функций И, ИЛИ, НЕ

x y	xy	x∨y
0 0	0	0
0 1	0	1
1 0	0	1
1 1	1	1

x	¬x
0	0
1	0

Их нужно знать!

Как проверить эти свойства???

Коммутативность

$xy \equiv yx$ $\quad x \vee y \equiv y \vee x$

Ассоциативность

$(xy)z \equiv x(yz)$ $\quad (x \vee y) \vee z \equiv x \vee (y \vee z)$

Дистрибутивность. Конъюнкция дистрибутивна относительно дизъюнкции как слева, так и справа, и наоборот: дизъюнкция дистрибутивна относительно конъюнкции как слева, так и справа

$x\,(y \vee z) \equiv xy \vee xz$ $\quad xy \vee z \equiv (x \vee y)\,(x \vee z)$

Законы Де Моргана

$\neg(xy) \equiv \neg x \vee \neg y$ $\quad \neg(x \vee y) \equiv \neg x\,\neg y$

Идемпотентность (поглощение)

$xx \equiv x$ $\quad x \vee x \equiv x$

Двойное отрицание

$\neg\neg x \equiv x$

Константы

$1x \equiv x$ $\quad 1 \vee x \equiv 1$ $\quad 0\,x \equiv 0$ $\quad 0 \vee x \equiv x$

Склеивание (абсорбция, поглощение)

$x\,y \vee x \equiv x$ $\quad (x \vee y)\,x \equiv x$

Проверка свойств двоичных функций

Проверим законы Де Моргана

$\neg(x\ y) \equiv \neg x \vee \neg y$

$\neg(x \vee y) \equiv \neg x\ \neg y$

(самостоятельно)

x y	xy	$\neg(xy)$	$\neg x$	$\neg y$	$\neg x \vee \neg y$
0 0	0	1	1	1	1
0 1	0	1	1	0	1
1 0	0	1	0	1	1
1 1	1	0	0	0	0

Пример: Взаимное исключение при использовании общего ресурса.

Пусть gi свойство: i-й процессор использует ресурс.

Условие корректности можно записать двумя эквивалентными способами:

ИЛИ $\neg(g1\ g2)$ *неверно, что и 1-й, и 2-й процессор используют ресурс*

ИЛИ $\neg g1 \vee \neg g2$ *или 1-й процессор не использует ресурс,*
или 2-й процессор не использует ресурс
или же ни тот, ни другой не использует ресурс

Проверка свойств двоичных функций

Проверим ассоциативность импликации:

$(x \Rightarrow y) \Rightarrow z \equiv ? \; x \Rightarrow (y \Rightarrow z)$

x y z	$x \Rightarrow y$	$(x \Rightarrow y) \Rightarrow z$	$y \Rightarrow z$	$x \Rightarrow (y \Rightarrow z)$
0 0 0	1	0	1	1
0 0 1	1	1	1	1
0 1 0	1	0	0	1
0 1 1	1	1	1	1
1 0 0	0	1	1	1
1 0 1	0	1	1	1
1 1 0	1	0	0	0
1 1 1	1	1	1	1

ДРУГИЕ СВОЙСТВА

$x\neg x \equiv 0$	$x\vee\neg x \equiv 1$	законы исключенного третьего
$(x\vee y)x \equiv x$	$(xy)\vee x \equiv x$	законы поглощения
$\neg 1 \equiv 0$	$\neg 0 \equiv 1$	свойства отрицания
$x\Leftrightarrow y \equiv (x\Rightarrow y)(y\Rightarrow x)$	$x\Rightarrow y \equiv \neg x\vee y$	свойства импликации
$x\Rightarrow 1 \equiv 1$	$x\Rightarrow 0 \equiv \neg x$	
$1\Rightarrow x \equiv x$	$0\Rightarrow x \equiv 1$	
$x\oplus y \equiv \neg xy \vee x\neg y$	$1\oplus x \equiv \neg x$	свойства сложения по модулю 2
$x\oplus(y\oplus x) \equiv y$		используется в шифровании

Хотя, строго говоря, обозначения

$\varphi\equiv\psi$ - ф-ии φ и ψ равносильны

$\varphi=\psi$ - ф-ия φ равна ф-ии ψ по определению

Вместо $\equiv$ часто пишут =

Применение XOR: шифрование-дешифрование одним и тем же ключом

Data:	M	O	S	C	O	W
ASCII Код:	1001101	1001111	1010011	1000011	1001111	1010111

Для шифрования используем ключ – конкретную строку

Key:	C	O	M	P	U	T
ASCII Код:	1000011	1001111	1001101	1010000	1010101	1010100

Получаем шифровку = Data ⊕Key

Шифровка:	0001110	0000000	0011110	0010011	0011010	0000011

Для дешифрования он использует ту же строку: Шифровка ⊕Key = Data

Data:	1001101	1001111	1010011	1000011	1001111	1010111

(Data ⊕Key) ⊕Key = Data поразрядная операция ⊕

Примеры представления двоичных функций в СДНФ

СДНФ – *Совершенная дизъюнктивная нормальная форма*:

$F(x_1, x_2, \ldots, x_n) = \mathcal{K}_1 \vee \mathcal{K}_2 \vee \mathcal{K}_3 \vee \ldots \vee \mathcal{K}_k$

$\mathcal{K}_i$ – конъюнкты

СДНФ – это дизъюнкция конъюнктов

$$K_i = x^{\sigma_x} y^{\sigma_y} z^{\sigma_z}$$

Каждый конъюнкт – это конъюнкция литералов

В каждый конъюнкт СДНФ каждая переменная входит с отрицанием или без

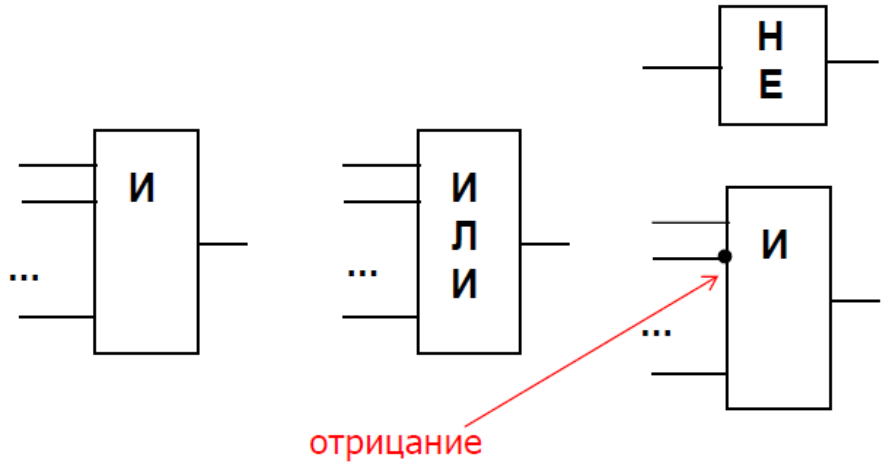
x y z	ite
0 0 0	0
0 0 1	1
0 1 0	0
0 1 1	1
1 0 0	0
1 0 1	0
1 1 0	1
1 1 1	1

Если функция не 0

ite (x, y, z) = x y z ∨ x y ¬z ∨ ¬x y z ∨ ¬x ¬y z

Значение основной теоремы и СДНФ

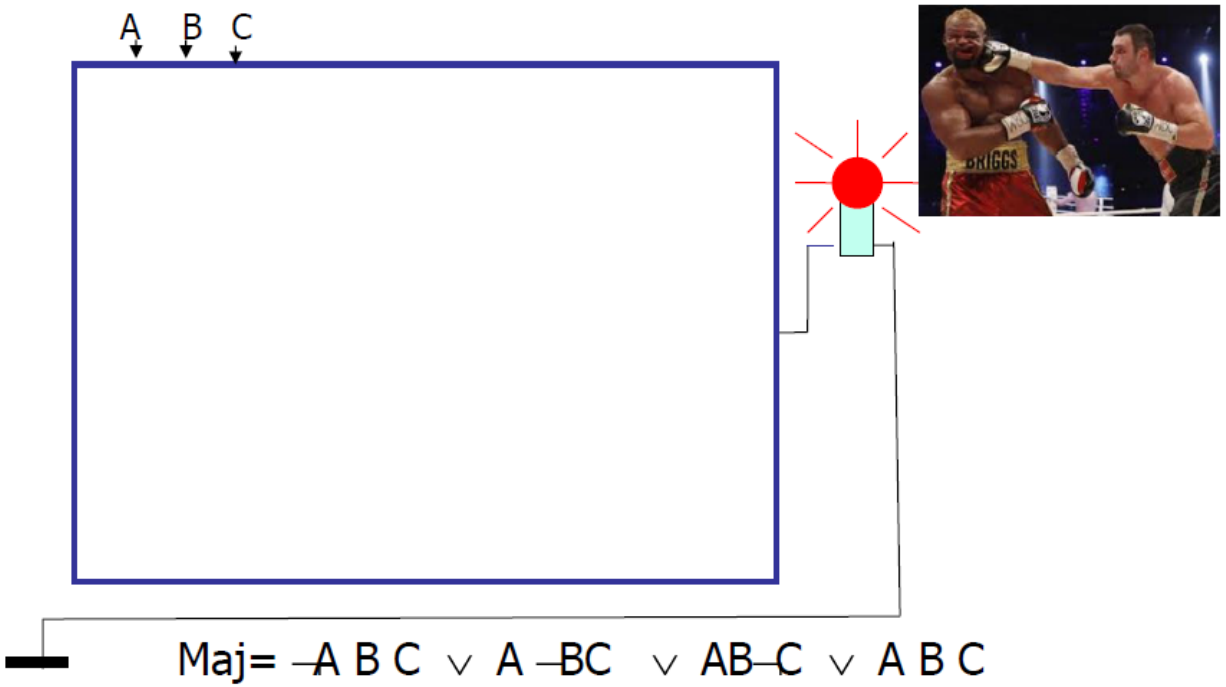
- Все двоичные функции, *от любого числа переменных*, можно представить единообразно, в виде дизъюнкции конъюнктов
- Такое представление для каждой функции единственно (с точностью до перестановки минтермов). Поэтому оно КАНОНИЧЕСКОЕ
- Если мы хотим реализовать ЛЮБУЮ двоичную функцию схемой, нам нужно иметь только три базовых элемента:
  - Реализующие И
  - Реализующие ИЛИ
  - Реализующие НЕ



Пример: мажоритарная схема принятия решений

- Три судьи судят боксерский поединок. Удар засчитывается, если хотя бы двое судей его засчитают (нажмут свою кнопку)

A B C	Maj
0 0 0	0
0 0 1	0
0 1 0	0
0 1 1	1
1 0 0	0
1 0 1	1
1 1 0	1
1 1 1	1

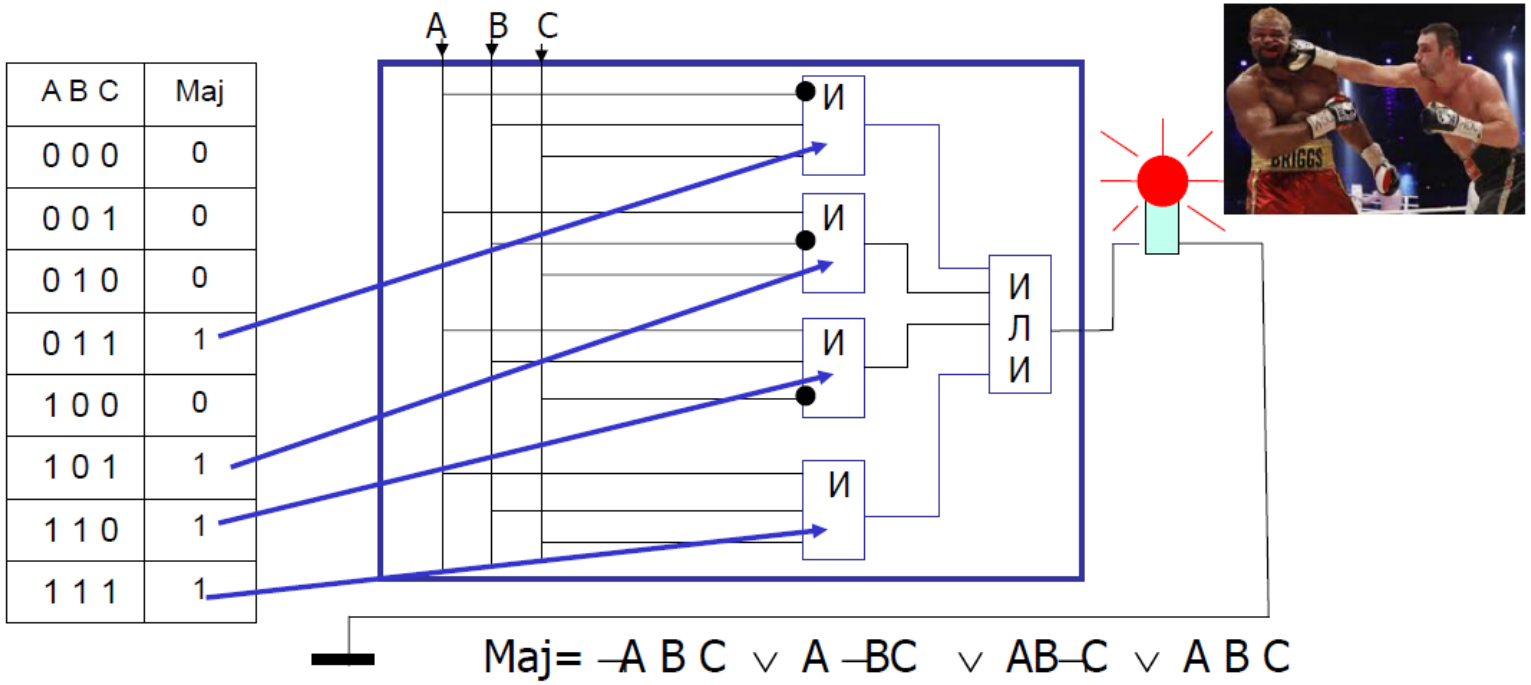


Как построить схему, реализующую правило принятия решений?

Пример: мажоритарная схема

- Три судьи судят боксерский поединок. Удар засчитывается, если хотя бы двое судей его засчитают (нажмут свою кнопку)

A B C	Maj
0 0 0	0
0 0 1	0
0 1 0	0
0 1 1	1
1 0 0	0
1 0 1	1
1 1 0	1
1 1 1	1



$$Maj = \bar{A}\,B\,C \vee A\,\bar{B}C \vee AB\,\bar{C} \vee A\,B\,C$$

Сложно, но главное то, что можем реализовать любую функцию.

Примеры представления двоичных функций в СДНФ

x y	xy	x∨y	x→y	x⊕y	x≡y	x|y	x↓y
0 0	0	0	1	0	1	1	1
0 1	0	1	1	1	0	1	0
1 0	0	1	0	1	0	1	0
1 1	1	1	1	0	1	0	0

$xy = xy;$

$x \vee y = \bar{x}y \vee x\bar{y} \vee xy$

$x \to y = \bar{x}\bar{y} \vee x\bar{y} \vee xy$

$x \oplus y = \bar{x}\,y \vee x\bar{y}$

$x \equiv y = \bar{x}\bar{y} \vee xy$

$x \mid y = \bar{x}\,\bar{y} \vee \bar{x}y \vee x\,\bar{y}$

$x \downarrow y = \bar{x}\,\bar{y}$

Схематическая реализация двоичных функций с помощью элементов И, ИЛИ, НЕ

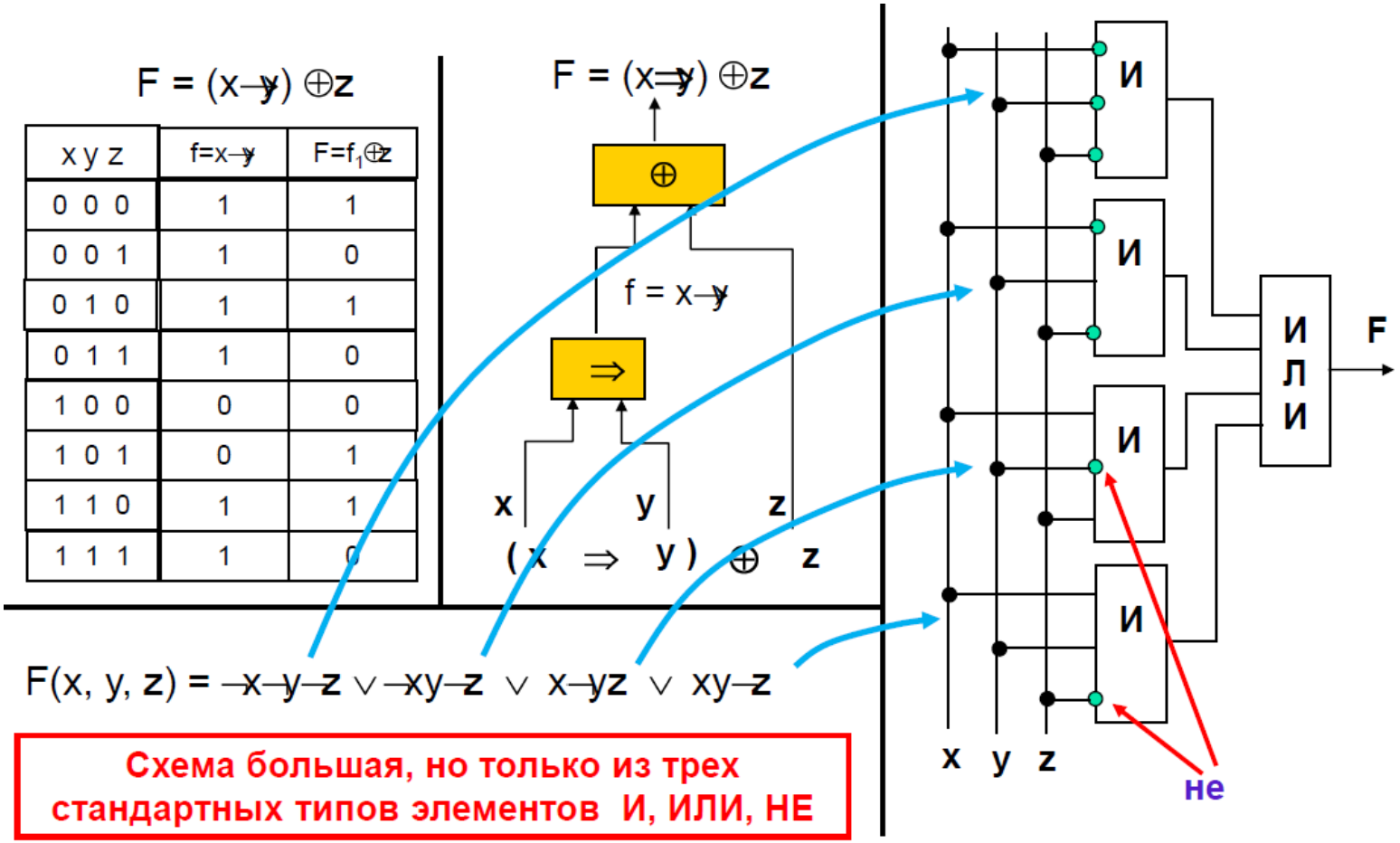


Схема большая, но только из трех стандартных типов элементов И, ИЛИ, НЕ

Что нужно помнить

Знать: 1. Все основные двоичные функции: НЕ, И, ИЛИ, $\oplus$, $\Rightarrow$ $\equiv$, $\downarrow$, |, ite

2. Свойства функций И, ИЛИ, НЕ
   коммутативность, ассоциативность, дистрибутивность, идемпотентность, формулы ДеМоргана

3. Как функции $\oplus$, $\Rightarrow$ $\equiv$, $\downarrow$, | представляются в базисе И, ИЛИ, НЕ

4. Смысл основной теоремы двоичных функций

Уметь: 1. Представить двоичные функции в виде ТИ и СДНФ

2. Строить ТИ любых функций от 2--5 аргументов по формулам

3. Проверять любые свойства булевых функций, например, коммутативность импликации или дистрибутивность $\oplus$ относительно $\Rightarrow$



$\neg$ЛЕКЦИЯ $\oplus$ ОТДЫХ